DT124A - Embedded Systems
Lab 1 & 2 Report

Gagan Adhikari

May 10, 2026

Authorship Statement

I *Gagan Adhikari* declare that this lab report is a result of my own original.
Explanation of Work: All of the task in both lab 1 and 2 was done by me and All of this reports is also done by me except for some syntax that I asked AI to create for the Latex. For the Labs I did use AI just to get verified my work atleast to be on a safe side before Labs and helped with flowcharts.

Contents

1 Introduction **4**

2 Materials and Methods **4**

2.1 Equipment 4

2.2 Methodology 4

3 LAB-1 **5**

3.1 Task 1: C Implementation 5

3.2 Task 2: Assembly Addition Loop 7

3.3 Data Comparison 8

3.4 Questions 8

3.5 Reflection 10

4 Task 1 – investigate the memory map when you debug the given code (Assembly) **11**

5 Task 2 – Compare Two Values and LED Control **16**

6 Task 3 – Arithmetic Operations and 7-Segment Display **20**

7 Task 4 – Find Maximum Value and Output 7-Segment Pattern **26**

8 Task 5 – 1 ms Delay Generation **28**

1 Introduction

In Lab 1 we have two task, we need to deal with. Task 1 is about writing a c program with adds two numbers and then debug the program using microchip stdio whereas in task 2 we where given a block of assembly code and to find out I/o map, execution time, Memory Map of different processors,

2 Materials and Methods

2.1 Equipment

- Microchip Studio IDE (Simulator)
- ATmega328p Microcontroller
- LEDs (Red and Green) and 220Ω Resistors
- 7-Segment Display

2.2 Methodology

The methodology followed the lab guidelines for Task 1 (C programming) and Task 2 (Assembly). Debugging was performed in step-mode within the simulator to monitor changes in the Status Register ($SREG$) and the I/O Map.

3 LAB-1

3.1 Task 1: C Implementation

As instructed during lectures, I have used microchip studio to write code and debug. As shown in Figure 1, I used basic libraries followed by the main function with the initialization of values and data types.And used addition operator to add.

```c
/*
 * C_Task_1.c
 *
 * Created: 4/12/2026 7:39:22 PM
 * Author : gagan
 */

#include <avr/io.h>
#include <stdlib.h>
#include <stdint.h>


int main(void)
{
    int x=4,y=5;
    int z;
    z= x+y;
    while (1)
    {
    }
}
```

Figure 1: C code

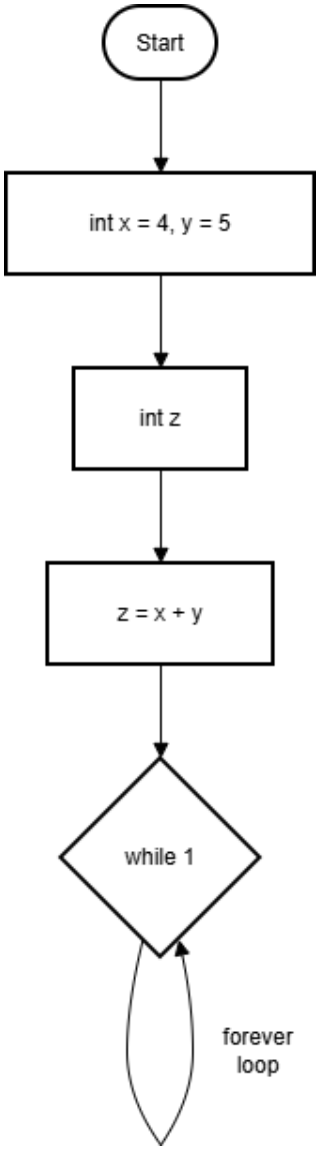


Figure 2: Flowchart for Task 1 lab 1

Figure: 2 flowchart shows the implementation process of the code for this task. starting from the start and forever loop at the end with while(1) condition which is always True.

Watch and Registers: I have created watch section to track variables and there addresses. Which shows updated values after every step over. AS you can see in the figure 3.

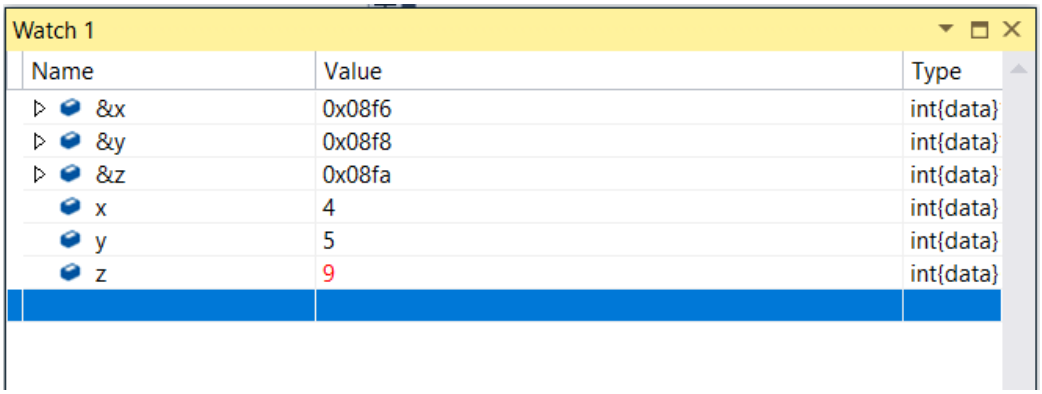


Figure 3: watch window

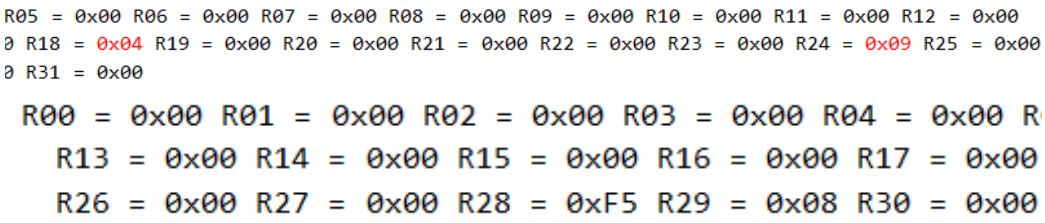


Figure 4: Register

In figure 4, We can check in which registers is our value saved/stored.

3.2 Task 2: Assembly Addition Loop

The program loads three registers with initial values (R16 = 3, R17 = 10, R18 = 2) and then copies R17 into R1 and R18 into R2 to prepare for repeated addition.As shown in the "figure 5", Inside the loop labeled START, the instruction ADD R1, R2 adds the value in R2 (which is 2) to R1 on every iteration, effectively accumulating the sum. The loop counter in R16 is decremented each time using DEC R16, and the BRNE START instruction causes the loop to repeat until R16 reaches zero. After completing three iterations, the final accumulated value in R1 is written to PORTD, and the program enters an infinite loop using RJMP FOREVER.

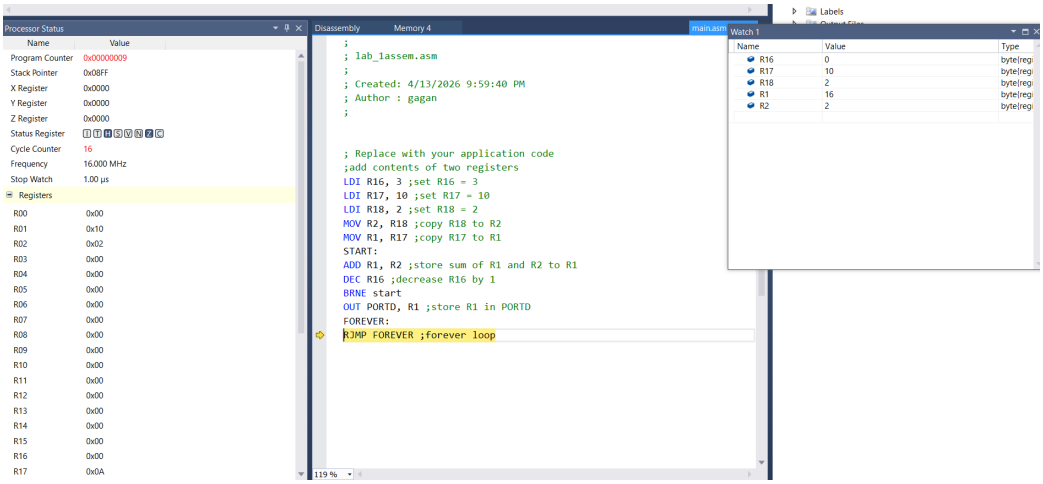


Figure 5: Step-by-step debugging of the Assembly loop.

3.3 Data Comparison

Below table: 1 shows the comparision of the source code size and machine code size of the task1 and task2. which shows task 1 have more source code size than task 2 code.....

Table 1: Comparison of Source vs. Machine Code Size

Program Type	Source Code Size (bytes)	Machine Code Size (bytes)
Task 1 (C)	974	48
Task 2 (Assembly)	438	32

3.4 Questions

1. Memory Map: The ATmega328p features 32 KB of Flash memory and 2 KB of SRAM. The Flash holds the program code while the SRAM stores data variables.

2. Execution Timing: for task 1: The code execution time was determined to be 625 us based on the total cycle count divided by the clock frequency of (16MHZ). Total cycle count was counted by counting instructions. for task 2: The code execution time was 16us

3. Status Register (*SREG*): The Status Register contains flags such as Zero (Z), Carry (C), and Negative (N). During the `DEC R16` instruction in Task 2, we observed the Z-flag being set once R16 reached zero.

3.5 Reflection

In task 1 I found out that forever loop is really neccessary if we want to make program run all the time in the avr programming, But what I had learned before we didn't use that. And finding source code size and machine code size was also abit tricky for me.But with the help from my friends and internet I was able to figure out all of these things. And for the Task2, everything was similar to our previous course with eric except for the code itself which was assembly. I find assembly is a bit longer and required a datasheet too but exciting also.

Introduction to Lab 2

The objective of Lab 2 was to develop further understanding of AVR assembly programming and hardware interfacing using the ATmega328p microcontroller. The lab focuses on comparisons, simple arithmetic in assembly, memory handling, 7-segment display control, and accurate delay generation.

All programs were written and tested in the Microchip Studio simulator before any hardware verification. Flowcharts are provided for each task and full assembly listings are included in the appendix as required.

4 Task 1 – investigate the memory map when you debug the given code (Assembly)

Objectives

The objective of this task is to practice basic AVR assembly operations involving register manipulation, arithmetic, and memory storage. Specifically, the program:

- Stores the computed results into SRAM using indirect addressing (X-pointer)
- Repeats the process for three different register pairs
- Allows debugging step-by-step in the simulator
- Demonstrates how register operations correspond to memory changes in SRAM

Theory

AVR microcontrollers use general-purpose registers (R0–R31) for arithmetic and data manipulation. Memory access is performed using:

- Direct addressing (e.g., `STS address, Rn`)
- Indirect addressing using pointer registers:
  - X = R27:R26
  - Y = R29:R28
  - Z = R31:R30

Key Concepts

1. Register Loading
`LDI` loads an immediate value (0–255) into a register.

2. Register-to-Register Move
`MOV Rd, Rr` copies data between registers.

3. Arithmetic
`ADD Rd, Rr` performs 8-bit addition and updates status flags (SREG).

4. Memory Storage

- Indirect store: `ST X+, Rn` (stores and increments pointer)
- Direct store: `STS address, Rn`

5. Debugging

- Observe register values changing
- Track X-pointer increment
- Monitor SRAM updates in memory view

Code Explanation

1. Load Values

```
LDI R17, 16
LDI R18, 32
```

Loads 16 into R17 and 32 into R18.

2. Copy Registers

```
MOV R1, R17
MOV R2, R18
```

Copies values into R1 and R2.

3. Addition

```
ADD R1, R2
```

R1 = 16 + 32 = 48.

4. Setup X-pointer

```
CLR R27
LDI R26, 80
```

Sets X = 0x0050.

5. Store First Result

```
ST X+, R19
STS 128, R1
```

Stores result in SRAM.

6. Second Addition

```
LDI R17, 127
LDI R18, 126
ADD R17, R18
```

R17 = 253.

7. Store Second Result

```
INC R26
ST X+, R17
```

8. Third Addition

```
LDI R19, 254
LDI R20, 2
ADD R19, R20
```

9. Store Third Result

```
INC R26
ST X+, R19
```

Implementation

1. Create a new AVR Assembly project in Microchip Studio.
2. Paste and compile the code.
3. Start simulation using Debug mode.
4. Step through the program using F11.

(a) Registers

Program Counter	0x00000000
Stack Pointer	0x08FF
X Register	0x0055
Y Register	0x0000
Z Register	0x0000
Status Register	I T H S V N Z C
Cycle Counter	16386
Frequency	16.000 MHz
Stop Watch	1,024.13 µs

(b) processor status

Name	Value
R17	253
R18	126
R1	48
R2	32
&x	0x0004
R19	0
R26	85

(c) Watch window

Figure 6: Observation for task-1

Observe: Observation is made from Figure: 6

- Register updates after each instruction
- X-pointer incrementing
- SRAM addresses updating
- Direct storage at address 0x0080

Conclusion

This experiment demonstrates how AVR assembly instructions manipulate registers and memory. It highlights indirect addressing using pointer registers and provides practical experience with debugging and memory analysis.

5 Task 2 – Compare Two Values and LED Control

Objective

Compare a register value with 50 and indicate the result using two LEDs: green for greater than 50, red for less than or equal to 50.

Theory

The program compares two unsigned 8-bit values using the AVR CP instruction, which updates SREG flags (Zero, Carry, Negative). Conditional branches BRLO and BREQ direct program flow so the code can choose between two output states on PORTB depending on whether $R19 > R18$ or $R19 \leq R18$.

Implementation

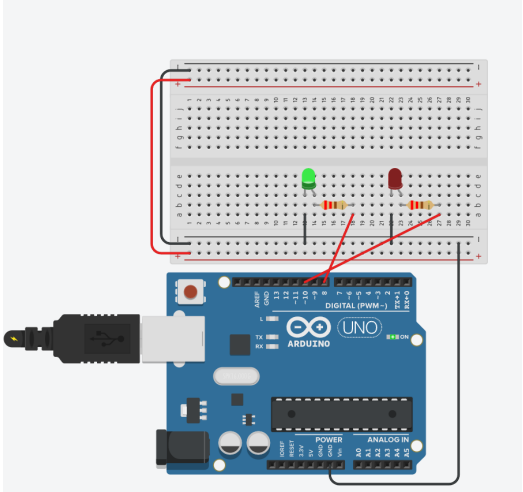


Figure 7: connection for task-2

Load the test value into R19 and the threshold (50) into R18, execute CP R19, R18, then use BRLO/BREQ to jump to the lower/equal case. If greater, set DDRB = 0x01 and PORTB = 0x01 to drive PB0 (green LED); otherwise set DDRB = 0x04 and PORTB = 0x04 to drive the alternate pin (red LED). The program ends in an infinite loop to hold the output state.

(See Appendix 8)

Flowchart

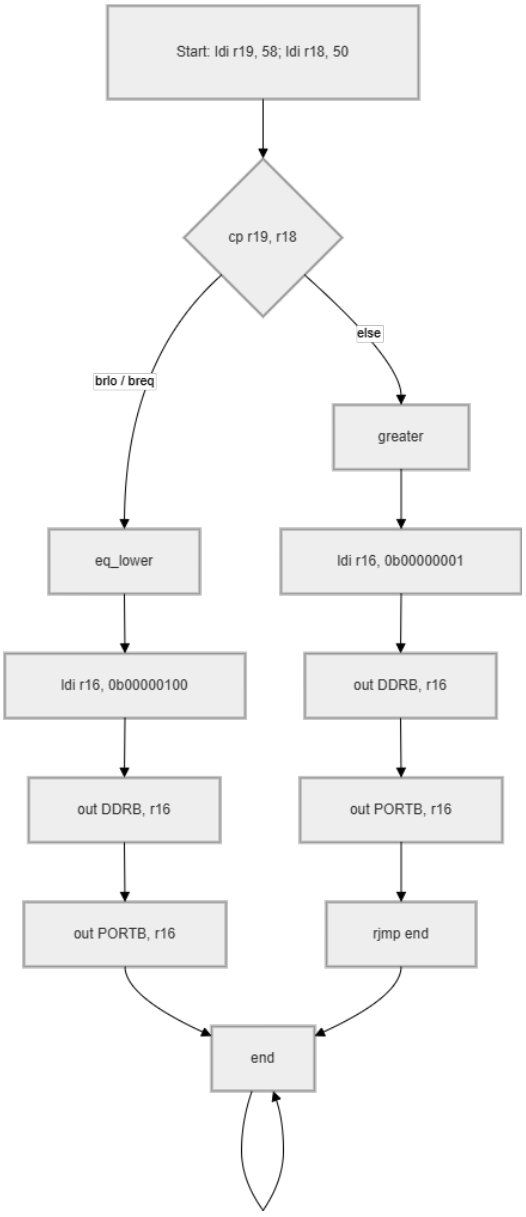


Figure 8: flowchart for task-2

The flowchart in figure 8 shows a simple decision-making program. It begins by loading two numbers into registers and comparing them. If the first number is less than or equal to the second, the program follows the lower/equal path and loads a specific output value into a register before sending it to the output port. If the first number is greater, it follows the greater path and

loads a different output value instead. The program then jumps to the end and stays there, keeping the chosen output active.

Result

When run in the simulator or on hardware, the code lights the green LED on PB0 if R19 is greater than 50; otherwise the red LED is lit. The decision is immediate after the compare and remains stable because the program loops indefinitely.

6 Task 3 – Arithmetic Operations and 7-Segment Display

Objective

Perform addition, multiplication and subtraction on small integers (results constrained to 0–9) and display the result on a single 7-segment display connected to `PORTD`.

Theory

As shown in figure 9, 7-segment display requires mapping decimal digits 0–9 to segment bit patterns. After computing the arithmetic result in registers, the result is compared against known values (0–9) and the corresponding segment pattern is written to `PORTD`.

Implementation

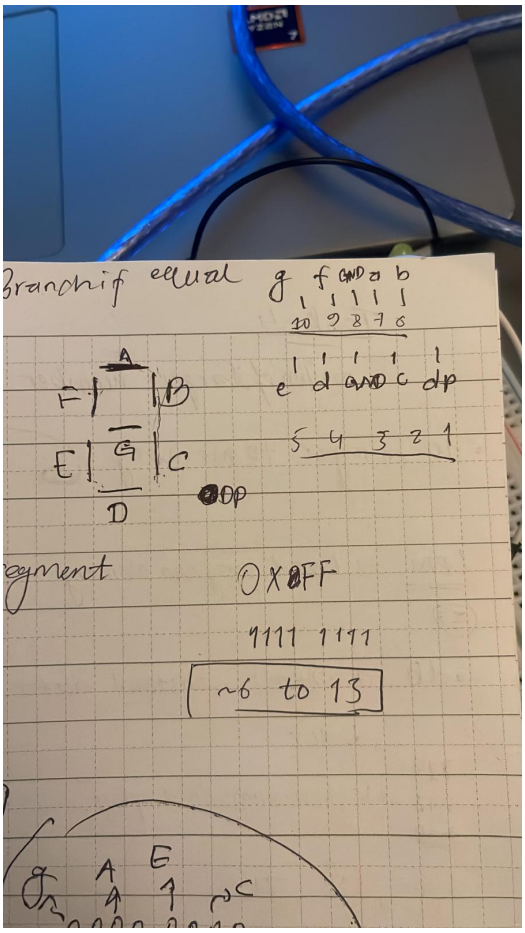


Figure 9: implementation of 7-seg display

1. Load operands into registers (e.g., `R17`, `R18`).
2. Perform the arithmetic operation (`ADD`, `MUL`, `SUB`).
3. Ensure the result is within 0–9.
4. Translate the numeric result to the 7-segment bit pattern and write to `PORTD`.
5. A 7-segment display is made of seven LED segments labeled A–G (and sometimes DP). Each segment is controlled individually by sending a HIGH or LOW signal depending on whether the display is common-anode

or common-cathode. To show a digit, you activate the correct combination of segments; for example, writing $0xFF$ or a binary pattern like 11111111 represents turning all segments on. The notes map each segment (A, B, C, D, E, F, G) to its corresponding bit position, showing how binary values correspond to physical segments. The diagram also shows how segment bits are grouped and how logic (like ANDing certain bits) determines which segments light up for a given number.

(See Appendix 8)

Flowchart

The flowchart in figure 10 and 11 shows how the program loads two numbers, adds them, and then checks the result against each digit from 0 to 9. For every comparison, a "yes" branch leads to a block that loads the correct 7-segment pattern and outputs it. A "no" branch continues to the next comparison. Once the matching digit is found and its pattern is displayed, the program moves to the end and stays in an infinite loop to keep the output stable.

I have another flowchart in my github `https://github.com/gAgAn-Adhikari/LAB_Images_other_riquired_Files.git`

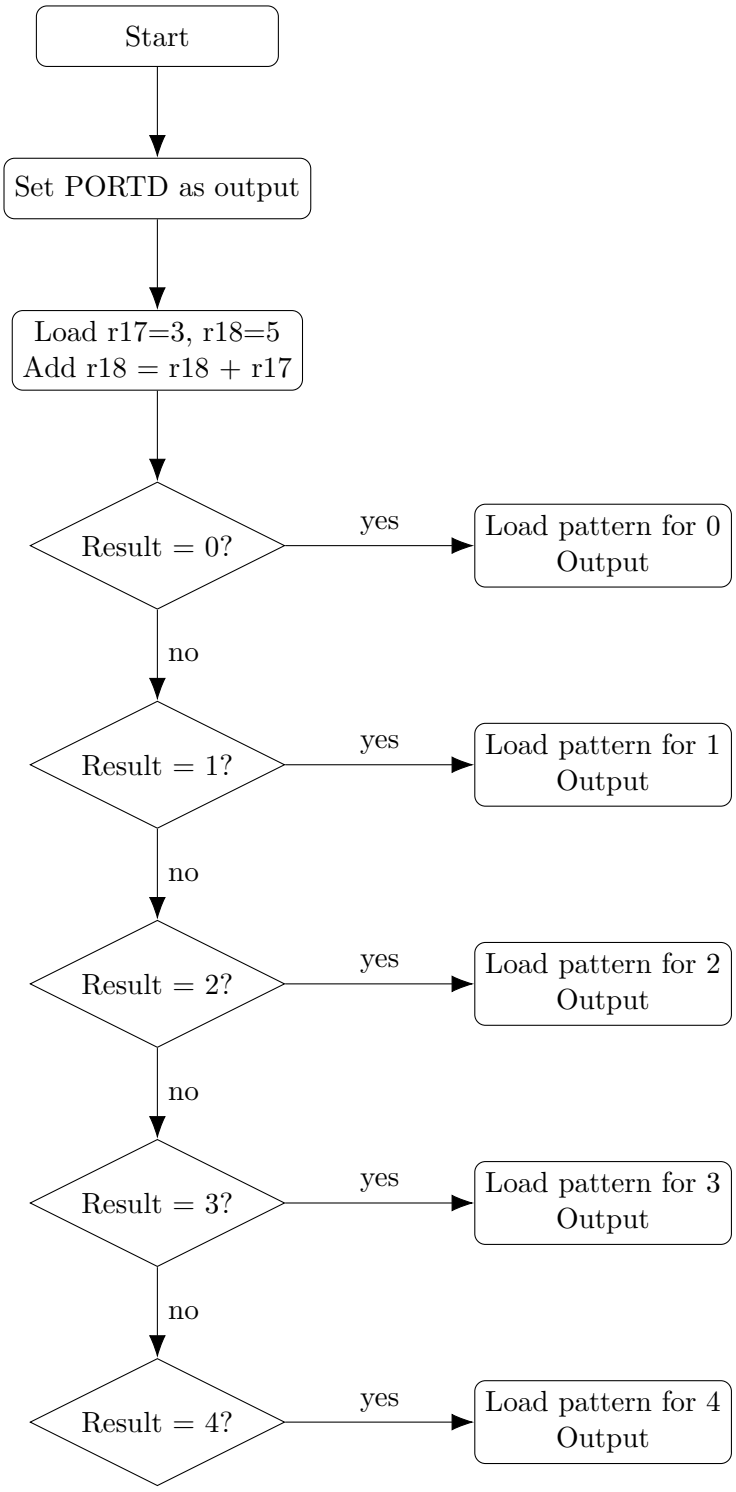


Figure 10: Flowchart – Part 1

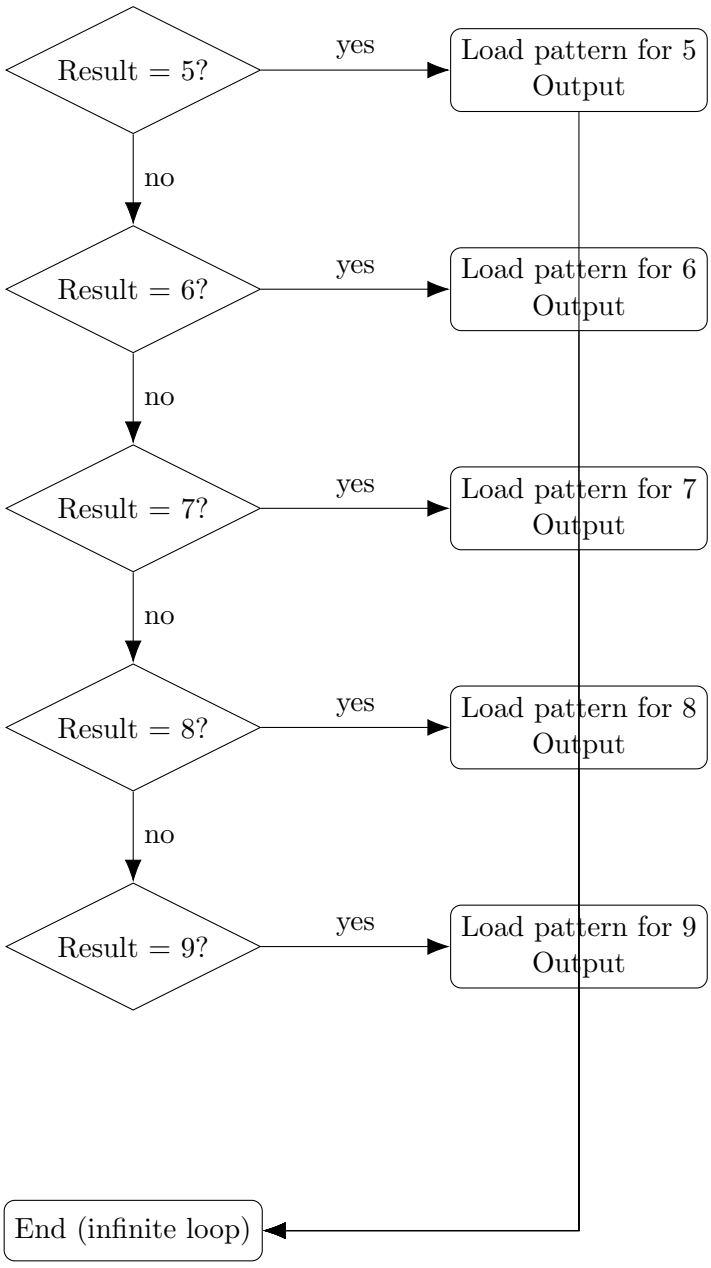


Figure 11: Flowchart – Part 2

Result

The simulator confirmed correct arithmetic results and correct segment patterns on `PORTD` for the tested values.

7 Task 4 – Find Maximum Value and Output 7-Segment Pattern

Objective

Identify the largest value from a set of values and output the corresponding 7-segment pattern on `PORTD`.

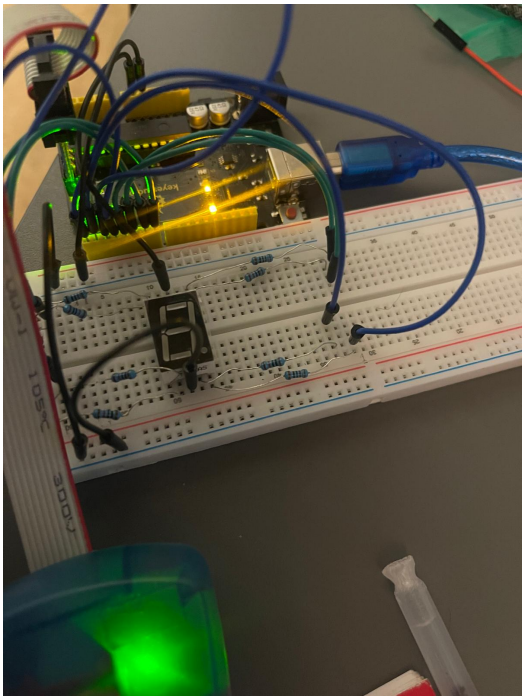


Figure 12: 7-segment Display

Theory

A running-maximum algorithm compares each sort of input value with the current maximum and updates the maximum and its associated output pattern when a larger value is found. For full credit, the lab expects a memory-based implementation (values in SRAM) using indirect addressing (X register). The provided implementation below uses immediate values for clarity; a memory-based variant is recommended for the final submission. So, to fullfill all the requirement, I have two codes one with basic logics with out using X registers. I will put my code with basic logic on GitHub `https://github.com/gAgAn-Adhikari/LAB_Images_other_riquired_Files.git`.

Implementation summary

- Store ten integer values in SRAM starting at address 0x0100.
- Store ten 7-segment patterns in SRAM starting at address 0x0200 in the same order.
- Scan the values using Z pointer, compare each value with the current maximum in r17.
- Update the maximum and its index in r19 when a larger value is found.
- Compute the pattern address by adding the index to the pattern base and load the pattern into r21.
- Output the pattern to PORTD to display the position of the maximum value.

(See Appendix 8)

Flowchart

Flowchart in github, click here

Result

As flowchart showsclick HereThe program stores ten data bytes in SRAM at 0x0100 and the corresponding 7-segment bit patterns at 0x0200, then scans the value list using the Z pointer; R17 holds the running maximum, R19 records the index of that maximum, and R20 is the running index during the loop. Each value is loaded with ld r18, Z+ and compared to r17, updating r17 and r19 when a larger value is found. After the loop the code computes the pattern address by adding the index to the pattern base, loads the matching pattern into R21, and writes R21 to PORTD. The result is that the 7-segment display shows the digit corresponding to the position of the largest value.

8 Task 5 – 1 ms Delay Generation

Objective

Create a software delay routine that produces an approximately 1 ms delay on a 16 MHz clocked ATmega328p.

Theory and calculation

Clock period:

$$T = \frac{1}{16MHz} = 62.5ns$$

Number of cycles for 1 ms:

$$\frac{1ms}{62.5ns} = 16000cycles$$

A nested loop with carefully counted instruction cycles is used to reach approximately 16000 cycles. The exact loop counts must be verified and tuned in the simulator.

Implementation

The nested-loop delay routine uses an inner loop of 252 iterations (R18) repeated 21 times by the outer loop (R17), followed by a block of nop instructions to fine-tune timing; this structure yields a deterministic software delay that can be tuned to 1.000 ms on a 16 MHz ATmega328p. Cycle accounting shows the nested loops produce the bulk of the 16,000 clock cycles required for 1 ms, and the trailing nops absorb the remaining cycles so the total matches the target when verified in the simulator. The routine is simple, repeatable, and easy to adjust by changing the ldi constants or the number of nops. As a result you can see my code in Appendix 8.

(See Appendix 8)

Flowchart

link to flowchart

For this flowchart It consists of 62 nops and flowchart shapes as per the instruction on the code like BRNE is inside a diamond as it is a conition.

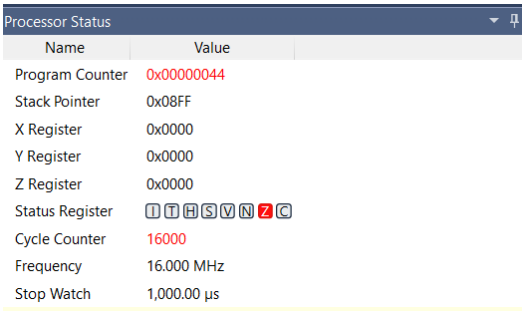


Figure 13: Watch-windows

Result

The delay routine uses nested loops (outer counter in R17, inner counter in R18) to produce a long software delay: the inner loop (dec r18 / brne l2) runs 252 iterations, and the outer loop repeats this 21 times, yielding a large, repeatable cycle count. Several nop instructions follow to fine-tune the final timing and absorb remaining cycles. When run on a 16 MHz AVR, this structure produces an approximately 1 ms delay per subroutine call after simulator verification and minor adjustments. The routine is simple, deterministic, and easy to tune by changing the ldi constants for precise timing.

I have 16k nop code in my GitHub too, with a flowchart(if you want to take a look).link to flowchart

Appendix

Appendix A – Assembly Code for Task 2

Listing 1: Task 2 – Compare and LED control

```
; Task 2: Compare R19 with 50 and light green  if greater, else red

start:
    ldi r19, 58
    ldi r18, 50

    cp r19, r18
    brlo eq_lower
    breq eq_lower

greater:
    ldi r16, 0b00000001
    out DDRB, r16
    out PORTB, r16
    rjmp end

eq_lower:
    ldi r16, 0b00000010
    out DDRB, r16
    out PORTB, r16

end:
    rjmp end
```

Appendix B – Assembly Code for Task 3

Listing 2: Task 3 – Arithmetic and 7-segment display

```
; Task 3: simple arithmetic and 7-segment output on PORTD

main:
    ldi r16, 0xFF
    out DDRD, r16        ; set PORTD as outputs for 7-seg

    ldi r17, 3           ; operand A
```

```
    ldi r18, 5          ; operand B
    add r18, r17        ; r18 = r18 + r17  (example: 3 + 5 = 8)

    ; compare result and branch to correct segment pattern
    cpi r18, 0
    breq zero
    cpi r18, 1
    breq one
    cpi r18, 2
    breq two
    cpi r18, 3
    breq three
    cpi r18, 4
    breq four
    cpi r18, 5
    breq five
    cpi r18, 6
    breq six
    cpi r18, 7
    breq seven
    cpi r18, 8
    breq eight
    cpi r18, 9
    breq nine

zero:
    ldi r16, 0b00111111
    out PORTD, r16
    rjmp end

one:
    ldi r16, 0b00010010
    out PORTD, r16
    rjmp end

two:
    ldi r16, 0b10111100
    out PORTD, r16
    rjmp end

three:
    ldi r16, 0b10110160
    out PORTD, r16
```

```
    rjmp end

four:
    ldi r16, 0b11010010
    out PORTD, r16
    rjmp end

five:
    ldi r16, 0b11100110
    out PORTD, r16
    rjmp end

six:
    ldi r16, 0b11101110
    out PORTD, r16
    rjmp end

seven:
    ldi r16, 0b00110010
    out PORTD, r16
    rjmp end

eight:
    ldi r16, 0b11111110
    out PORTD, r16
    rjmp end

nine:
    ldi r16, 0b11110110
    out PORTD, r16
    rjmp end

end:
    rjmp end
```

Appendix C – Assembly Code for Task 4

Listing 3: Task 4 – Find maximum value and output position of maximum number in 7-seg pattern

```
; Task 4: Find the largest of a set of immediate values and output its 
ldi r16, 0x08
```

```
out SPH, r16
ldi r16, 0xFF
out SPL, r16

ldi r16, 0xFF
out DDRD, r16

ldi r30, 0x00
ldi r31, 0x01

ldi r16, 0
st Z+, r16

ldi r16, 20
st Z+, r16

ldi r16, 2
st Z+, r16

ldi r16, 5
st Z+, r16

ldi r16, 177
st Z+, r16

ldi r16, 20
st Z+, r16

ldi r16, 44
st Z+, r16

ldi r16, 55
st Z+, r16

ldi r16, 23
st Z+, r16

ldi r16, 0
st Z+, r16

ldi r30, 0x00
ldi r31, 0x02
```

```
ldi r16, 0b01111110
st Z+, r16

ldi r16, 0b00010010
st Z+, r16

ldi r16, 0b10111100
st Z+, r16

ldi r16, 0b10110110
st Z+, r16

ldi r16, 0b11010010
st Z+, r16

ldi r16, 0b11100110
st Z+, r16

ldi r16, 0b11101110
st Z+, r16

ldi r16, 0b00110010
st Z+, r16

ldi r16, 0b11111110
st Z+, r16

ldi r16, 0b11110110
st Z+, r16

ldi r30, 0x00
ldi r31, 0x01

ldi r17, 0
ldi r19, 0
ldi r20, 0
ldi r22, 10

LOOP:
ld r18, Z+
cp r18, r17
brlo SKIP
```

```
mov r17, r18
mov r19, r20

SKIP:
inc r20
dec r22
brne LOOP

ldi r30, 0x00
ldi r31, 0x02

add r30, r19
adc r31, r1

ld r21, Z
out PORTD, r21

END:
rjmp END
```

Appendix D – Assembly Code for Task 5

Listing 4: Task 5 – Nested-loop delay example

```
; Task 5: Generate approximately 1 ms delay on 16 MHz clock using nested
; Approach: use two nested loops. Outer counter in r17, inner counter in
; Each inner loop iteration consumes a known number of cycles; outer loo

ldi r17, 21
l1:
ldi r18, 252
l2:
dec r18
brne l2
dec r17
brne l1
nop
nop
nop
nop
nop
nop
```

```
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
```

```
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
nop
```